

Android 第 3 章 —— 2 常用布局教学讲义

各位同学，上节课咱们初步认识了常用布局，这节课咱们深入拆解最核心的几种布局：LinearLayout（线性布局）、RelativeLayout（相对布局）、GridLayout（网格布局），还有能解决界面装不下问题的滚动视图。全程大白话，保证大家听完就能用。

一、LinearLayout：最“规矩”的排队布局

LinearLayout（线性布局）就像咱们排队，要么“横着排”，要么“竖着排”，没有其他花里胡哨的排列方式，是咱们最常用的布局之一。

1. 控制排列方向：horizontal（横排）和 vertical（竖排）

线性布局的核心属性是`android:orientation`，它决定了控件是“横着站”还是“竖着站”：

- **horizontal（水平排列）**：控件从左到右依次排列，就像大家并排站成一行。比如在布局文件里写`android:orientation="horizontal"`，里面的按钮、文本控件就会从左到右排开。这里要注意，水平排列时，控件的`layout_width`尽量别设成`match_parent`（占满行），不然一个控件就占满整行，后面的控件就没地方放了。
- **vertical（垂直排列）**：控件从上到下依次排列，像大家竖着站成一列。在布局文件里写`android:orientation="vertical"`，控件就会从上到下逐个摆放。同理，垂直排列时，控件的`layout_height`别轻易设`match_parent`，避免一个控件占满整列。

举个例子，想让两个按钮水平排列，布局代码大概是这样：

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="按钮1"/>
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
        android:text="按钮2"/>
    </LinearLayout>
```

运行后就能看到两个按钮并排放在一行了。

2. 线性布局的权重：layout_weight，让控件“按比例分地盘”

有时候咱们想让控件按比例占用屏幕空间，比如让两个文本框各占屏幕宽度的一半，这时候就需要layout_weight（权重）属性。权重就像“分蛋糕的比例”，数值越大，占的空间越多。

（1）权重的核心逻辑

- 权重值是相对的，比如两个控件的权重分别是 1 和 1，那它们就各占剩余空间的 1/2；如果权重是 2 和 1，那前者占 2/3，后者占 1/3。
- 用权重时，要把对应方向的尺寸设为0dp（水平排列设layout_width="0dp"，垂直排列设layout_height="0dp"），这样权重才能完全生效，不然控件会先按自己的尺寸占空间，剩下的才按权重分，容易出问题。

（2）实例：两个控件各占一半宽度

比如让两个文本控件水平排列，各占屏幕宽度的一半，代码如下：

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">
    <TextView
        android:layout_width="0dp" <!-- 水平排列，宽度设0dp -->
        android:layout_height="wrap_content"
        android:layout_weight="1" <!-- 权重1 -->
        android:text="我占一半"/>
    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1" <!-- 权重1 -->
        android:text="我也占一半"/>
</LinearLayout>
```

这里两个控件权重都是 1，所以各分一半宽度；如果把第一个的权重改成 2，第二个还是 1，那第一个就占 2/3，第二个占 1/3。

二、RelativeLayout：最“灵活”的相对布局

RelativeLayout（相对布局）比线性布局灵活多了，它可以让控件“相对于其他控件”或“相对于父布局”来定位，比如“按钮 A 在按钮 B 的右边”“文本框在父布局的右下角”，适合做复杂的界面布局。

它的核心是“相对属性”，常用的有两类：

1. 相对于父布局定位（以父布局为参考）

这类属性让控件相对于整个相对布局（父布局）的位置来放，常用的有：

- `android:layout_centerInParent="true"`：控件在父布局正中间（水平 + 垂直居中）。
- `android:layout_alignParentLeft="true"`：控件靠父布局左边。
- `android:layout_alignParentRight="true"`：控件靠父布局右边。
- `android:layout_alignParentTop="true"`：控件靠父布局顶部。
- `android:layout_alignParentBottom="true"`：控件靠父布局底部。

比如想让一个按钮放在父布局的右下角，代码是：

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="200dp">
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentRight="true"
        android:layout_alignParentBottom="true"
        android:text="右下角按钮"/>
</RelativeLayout>
```

2. 相对于其他控件定位（以兄弟控件为参考）

这类属性需要先给目标控件起个 “ID”（用`android:id="@+id/控件名"`），然后其他控件以这个 ID 为参考来定位，常用的有：

- `android:layout_toLeftOf="@id/目标控件ID"`：控件在目标控件的左边。
- `android:layout_toRightOf="@id/目标控件ID"`：控件在目标控件的右边。
- `android:layout_above="@id/目标控件ID"`：控件在目标控件的上方。
- `android:layout_below="@id/目标控件ID"`：控件在目标控件的下方。
- `android:layout_alignTop="@id/目标控件ID"`：控件的顶部和目标控件的顶部对齐。

比如想让 “按钮 2” 在 “按钮 1” 的右边，代码如下：

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <!-- 先给按钮1设ID -->
    <Button
        android:id="@+id/btn1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="按钮1"/>
    <!-- 按钮2在按钮1右边 -->
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_toRightOf="@id/btn1"
        android:text="按钮2"/>
</RelativeLayout>
```

这样运行后，按钮 2 就会紧跟在按钮 1 的右边。

三、GridLayout：最 “整齐” 的网格布局

GridLayout（网格布局）就像咱们用的表格，把界面分成若干行和列，每个控件放在 “单元格” 里，适合排列整齐的元素，比如计算器的按键、图片墙。

它的核心是控制 “行数” 和 “列数”，还有控件占的 “单元格数量”。

1. 控制行数和列数：rowCount 和 columnCount

- **columnCount (列数)**：设置网格布局有多少列，比如`android:columnCount="3"`，就是分成 3 列。
- **rowCount (行数)**：设置网格布局有多少行，比如`android:rowCount="2"`，就是分成 2 行。

比如想做一个 2 行 3 列的网格，放 6 个按钮，代码如下：

```
<GridLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:columnCount="3" <!-- 3列 -->
    android:rowCount="2"> <!-- 2行 -->
    <Button android:text="1"/>
    <Button android:text="2"/>
    <Button android:text="3"/>
    <Button android:text="4"/>
    <Button android:text="5"/>
    <Button android:text="6"/>
</GridLayout>
```

运行后就能看到 6 个按钮整齐地排成 2 行 3 列，每个按钮占一个单元格。

2. 让控件跨多行 / 多列（可选）

有时候咱们想让一个控件占多个单元格，比如让某个按钮占 2 列，这时候可以用：

- `android:layout_columnSpan="2"`：控件跨 2 列（横向占 2 个单元格）。
- `android:layout_rowSpan="2"`：控件跨 2 行（纵向占 2 个单元格）。

比如在上面的 2 行 3 列网格里，让第一个按钮占 2 列，代码修改如下：

```
<GridLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:columnCount="3"
    android:rowCount="2">
    <!-- 第一个按钮跨2列 -->
```

```
<Button
    android:layout_columnSpan="2"
    android:text="1（占2列）"/>
<Button android:text="2"/>
<Button android:text="3"/>
<Button android:text="4"/>
<Button android:text="5"/>
</GridLayout>
```

这样第一个按钮就会占第一行的前 2 个单元格，后面的按钮会自动调整位置。

四、滚动视图：解决“界面装不下”的问题

有时候界面内容太多，屏幕显示不下，比如长文本、很多控件，这时候就需要“滚动视图”，让用户能上下或左右滑动查看全部内容。常用的滚动视图有两种：ScrollView（垂直滚动）和 HorizontalScrollView（水平滚动）。

1. ScrollView：垂直滚动（上下滑）

ScrollView 只能包含一个**直接子控件**（通常是 LinearLayout），所有要滚动的内容都放在这个子控件里，它会让内容在垂直方向上滚动。

比如想让一长串文本能上下滑动查看，代码如下：

```
<ScrollView
    android:layout_width="match_parent"
    android:layout_height="200dp"> <!-- 限定滚动区域高度 -->
    <!-- 子控件用LinearLayout，里面放文本 -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">
        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="这里是很长很长的文本...（省略几百字）...看完需要上下滑动"/>
    </LinearLayout>
</ScrollView>
```

这里给 ScrollView 设了固定高度（200dp），当文本超过这个高度时，就会出现垂直滚动条，用户可以上下滑动看全部内容。

2. HorizontalScrollView：水平滚动（左右滑）

和 ScrollView 类似，HorizontalScrollView 也只能有一个直接子控件，不过它是让内容在**水平方向**上滚动，适合内容太宽、屏幕装不下的情况，比如横向排列的多个图片。

比如想让多个图片能左右滑动查看，代码如下：

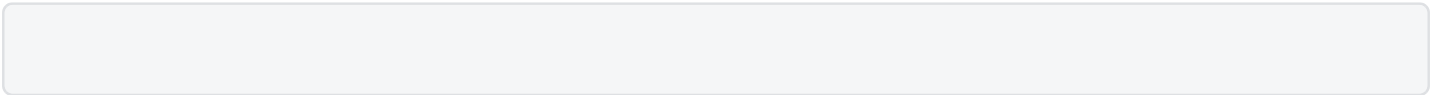
```
<HorizontalScrollView
    android:layout_width="match_parent"
    android:layout_height="100dp"> <!-- 限定滚动区域高度 -->
    <!-- 子控件用水平排列的LinearLayout -->
    <LinearLayout
        android:layout_width="wrap_content" <!-- 宽度随内容变，才会滚动 -->
        android:layout_height="match_parent"
        android:orientation="horizontal">
        <ImageView
            android:layout_width="150dp"
            android:layout_height="match_parent"
            android:src="@drawable/img1"/>
        <ImageView
            android:layout_width="150dp"
            android:layout_height="match_parent"
            android:src="@drawable/img2"/>
        <ImageView
            android:layout_width="150dp"
            android:layout_height="match_parent"
            android:src="@drawable/img3"/>
    </LinearLayout>
</HorizontalScrollView>
```

这里 LinearLayout 的宽度设为 wrap_content（随图片总宽度变化），当图片总宽度超过屏幕宽度时，就会出现水平滚动条，用户可以左右滑动看所有图片。

五、布局使用小技巧

1. **优先用线性布局**：简单界面（比如一行按钮、一列文本）用 LinearLayout，代码简单，容易维护。

- 2. **复杂界面用相对布局**：需要控件相互参考定位时，用 RelativeLayout，少嵌套布局（嵌套太多会影响性能）。
- 3. **整齐排列用网格布局**：像计算器、图片墙这种规则的网格，用 GridLayout 最方便。
- 4. **内容太多用滚动视图**：不管是垂直还是水平，只要内容超出屏幕，就加对应的滚动视图，别让内容被截断。



（注：文档部分内容可能由 AI 生成）